

Received 30 September 2025, accepted 6 November 2025, date of publication 17 November 2025,
date of current version 21 November 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3633664

RESEARCH ARTICLE

YOLO-FES: An Improved Elephant Intrusion Detector Based on YOLOv8n

MUHAMMAD DANIAL MOHAMAD RIZWAN¹,
MD. AKBAR HOSSAIN², (Senior Member, IEEE), ATHIRAH NAJIHAH ZULKIFILI³,
JALAL UDDIN MD AKBAR¹, LAVINDAR SINGH⁴,
AND SYAFIQ FAUZI KAMARULZAMAN¹, (Senior Member, IEEE)

¹Faculty of Computing, Universiti Malaysia Pahang Al-Sultan Abdullah, Pekan, Pahang 26600, Malaysia

²School of Computing, Eastern Institute of Technology, Napier 4112, New Zealand

³Faculty of Manufacturing and Mechatronic Engineering Technology, Universiti Malaysia Pahang Al-Sultan Abdullah, Pekan, Pahang 26600, Malaysia

⁴Satok Bridge Digital Sdn. Bhd., TEGAS Digital Village, Kuching, Sarawak 93350, Malaysia

Corresponding authors: Md. Akbar Hossain (ahossain@eit.ac.nz) and Syafiq Fauzi Kamarulzaman (syafiq29@ump.edu.my)

This work was supported by Malaysia's Ministry of Higher Education through the Industrial Matching Program Grant (IMAP) under Grant IMaP/2/2024/ICT02/UMPSA/1.

This work involved human subjects or animals in its research. Approval of all ethical and experimental procedures and protocols was granted by Malaysia's Department of Wildlife and National Park (Permit No: JPHL&TN(IP):100-34/1.24 Jld 1729) with collaboration and consultation through the organization of Management and Ecology of Malaysian Elephant (MEME).

ABSTRACT Human–elephant conflict (HEC) poses a significant threat to both biodiversity and rural livelihoods, necessitating innovative monitoring solutions that are both accurate and deployable in resource-constrained environments. Existing deep learning models often trade off detection accuracy for computational efficiency, limiting their real-time use on edge devices. To address this challenge, we propose YOLO-FES, a lightweight object detection model tailored for efficient and accurate elephant intrusion detection. YOLO-FES integrates three key components: (i) a FasterNet block (F) that replaces bottleneck blocks in the backbone, (ii) an Efficient Multi-scale Attention (E) module to enhance feature representation, and (iii) a Slim-neck (S) powered by GSConv for improved feature fusion. The model was trained on a diverse dataset comprising trap cameras, CCTV, drones, mobile phones, television footage, and supplemented with elephant images from COCO2017. Experimental results demonstrate that YOLO-FES reduces parameters, FLOPs, and model size by 19%, 23.5%, and 16.1%, respectively, compared to YOLOv8n, while achieving higher accuracy with +1.5% mAP@0.5 and +1.2% mAP@0.5–0.95. Edge deployment evaluations confirm real-time performance, with inference times ranging from 24.7 ms on Jetson Orin Nano to 254.2 ms on Raspberry Pi 4B. These results establish YOLO-FES as a robust, low-cost, and deployable solution for real-time elephant intrusion detection, contributing to sustainable mitigation of human–elephant conflict.

INDEX TERMS Elephant intrusion detection, lightweight, YOLO, attention mechanism.

I. INTRODUCTION

The coexistence of humans and wildlife has long been a critical ecological and socioeconomic concern, particularly in regions where human activities overlap with natural habitats. Among large terrestrial mammals, elephants play a vital role in maintaining biodiversity and ecosystem balance. However, habitat fragmentation, agricultural expansion, and increasing human settlements have intensified Human-Elephant

Conflict (HEC), leading to substantial property damage, crop losses, and even fatalities. These conflicts are especially severe in regions such as India, Malaysia, and Sri Lanka, where human settlements encroach upon elephant migration corridors [1]. Addressing HEC is not only crucial for biodiversity conservation but also essential for safeguarding human livelihoods.

Historically, efforts to reduce HEC have relied on traditional techniques, including ground surveys, dung analysis, and manual patrolling. However, these approaches suffer from limitations such as high labor costs, environmental

The associate editor coordinating the review of this manuscript and approving it for publication was Abdel-Hamid Soliman¹.

constraints, and delays in response time [2]. Progress in artificial intelligence (AI) and Deep Learning (DL) provides groundbreaking opportunities to overcome these constraints. Instruments such as automated camera traps, geospatial analysis, and passive acoustic monitoring systems allow researchers to track the movements and behaviors of elephants with unmatched precision and scope. As an illustration, deep learning techniques have been effectively utilized to create real-time video surveillance systems to prevent clashes between people and elephants on train tracks [3]. Similarly, AI models utilizing bioacoustics have demonstrated effectiveness in detecting elephant sounds in real-time, supporting the creation of early warning systems designed to reduce conflict risks [4].

Recent developments, particularly Convolutional Neural Networks (CNNs), have significantly improved the accuracy of elephant detection and monitoring. In Sri Lanka, CNN-inspired models attained 94% accuracy in detecting elephants from visual datasets, thereby providing a scalable solution for real-time surveillance over large areas [5]. Additionally, monitoring with satellite collars has revealed distinct spatial and behavioral patterns in both conflict and non-conflict elephants, thereby providing valuable insights into conflict mitigation strategies [6]. These developments, along with traditional ecological knowledge, offer distinctive opportunities for understanding elephant behavior and creating adaptive conservation strategies.

Despite these technological advancements, some difficulties still exist. Numerous AI-driven solutions require extensive data gathering, powerful computational infrastructure, and substantial financial support, which limits their implementation in resource-constrained regions. Furthermore, fragmented habitats and the growing unpredictability of elephant movements hinder efforts to resolve conflicts. For example, more than 70% of the habitat of Asian elephants currently exists beyond protected areas, increasing the need for creative and flexible conservation approaches [7].

At the same time, existing lightweight object detection approaches also present notable limitations. YOLO-based detectors such as YOLOv5n [8] and YOLOv7-tiny [9] achieve efficiency but often at the expense of accuracy in challenging environments. Wildlife-specific frameworks, such as WilDect-YOLO [10], provide high detection accuracy; however, their heavy architectures limit their feasibility for real-time deployment on edge devices. Moreover, elephant detection studies [2], [4] primarily rely on high-performance computing and do not optimize specifically for lightweight edge scenarios. These gaps motivate the design of YOLO-FES.

To address these limitations, this study presents YOLO-FES, a lightweight yet high-precision object detection model designed to address the dual challenge of accurate and computationally efficient elephant intrusion detection. Built upon the YOLOv8n framework, YOLO-FES introduces architectural advancements in feature extraction, attention mechanisms, and feature fusion to enhance detection

performance while optimizing computational efficiency. A key distinguishing factor of YOLO-FES is its single-class specialization, which enables targeted optimization for elephant detection in real-world environments. This focused approach not only improves detection accuracy and robustness but also ensures the model's practical viability for deployment on edge devices in resource-constrained settings. By providing real-time, energy-efficient monitoring capabilities, YOLO-FES contributes to mitigating Human-Elephant Conflict (HEC)—a critical conservation challenge—while facilitating scalable and automated wildlife monitoring solutions.

The main contributions of this study are summarized as follows:-

- 1) **Improved Feature Extraction:** The bottleneck block in the standard C2f module is substituted with the FasterNet block, which employs partial convolution (PConv) for effective spatial feature extraction. Moreover, the Efficient Multi-scale Attention (EMA) module was incorporated to improve the model's ability to identify elephants at various scales, including situations where they are partially obscured and in diverse surroundings, while ensuring minimal computational expenses and high detection precision.
- 2) **Improved Feature Fusion:** The novel Slim-neck design was utilized, integrating VoV-GSCSP, GSBottleneck, and GSConv. This design significantly improved detection precision while reducing computational overhead.
- 3) **Performance Assessment:** Thorough comparisons with previous state-of-the-art models demonstrated the effectiveness of the YOLO-FES. It decreased parameters, FLOPs, and model size by 19%, 23.46% and 16.13%, respectively, while improving mAP50 and mAP50-95 by 1.5% and 1.2%, respectively, compared to the baseline model.

The remainder of this paper is organized as follows: Section II reviews related works, highlighting advances in conventional and AI-based elephant detection methods. Section III details the proposed YOLO-FES architecture, including its improvements compared to YOLOv8n. Section IV describes the experimental setup and presents a comprehensive analysis of results, including comparisons with existing models. Finally, Section VI discusses the implications of our findings and outlines potential future research directions.

II. RELATED WORKS

A. CONVENTIONAL METHODS FOR ELEPHANT DETECTION

The detection and monitoring of elephants have traditionally relied on manual and sensor-based methods such as camera traps, aerial surveillance, acoustic detection, and manual patrolling. While these approaches have been widely adopted for conservation efforts, they present significant challenges in scalability, automation, and real-time responsiveness.

- 1) **Camera Trap:** Camera traps are widely used for non-intrusive wildlife monitoring, triggering image capture based on motion. However, limited field of view, environmental dependencies, and labor-intensive image analysis restrict their effectiveness for large-scale deployment [11].
- 2) **Aerial Surveillance:** Elephant populations in open spaces, such as savannahs, can be quickly assessed using aircraft-based or drone-based surveys that cover vast areas. Although they cover a lot of ground, they are costly, weather-dependent, and prone to human error in counting due to fatigue [12], [13].
- 3) **Acoustic Detection:** Passive Acoustic Monitoring (PAM) systems capture elephant vocalizations, particularly in dense forests where visual detection is difficult. Although effective for non-visual environments, detection is limited to vocalization periods and is hampered by background noise [2].
- 4) **Seismic Detection:** This technique uses ground vibrations to detect elephant movements or low-frequency calls, which are effective in visually obstructed areas [14]. However, scalability is limited by manual frequency analysis, and the detection accuracy is affected by soil variations and environmental noise.
- 5) **Manual Patrol:** Manual patrols involve rangers observing and recording the presence, tracks, or signs of elephants in designated areas, providing detailed data in inaccessible regions. However, they are labor-intensive, limited in coverage, prone to human fatigue, and are affected by observer bias and environmental challenges.

B. EVOLUTION OF AI IN WILDLIFE DETECTION

The application of computer vision and deep learning to wildlife monitoring has evolved over the past decade, transitioning from traditional machine learning techniques to advanced deep neural networks. Early approaches relied on handcrafted feature-based machine learning models, such as Support Vector Machines (SVM), Random Forests, and Decision Trees, to classify elephants from images [15]. While these methods provided a foundation for automated detection, they lacked robust feature extraction capabilities, resulting in poor generalization to real-world, unconstrained environments.

With the advent of deep learning, particularly Convolutional Neural Networks (CNNs), significant improvements in detection accuracy and robustness were achieved. Models such as Faster R-CNN, Single Shot MultiBox Detector (SSD), and You Only Look Once (YOLO) have emerged as state-of-the-art object detection frameworks, enabling real-time tracking and classification of wildlife [16]. However, many deep learning models were developed for general object detection tasks, leading to computational inefficiencies when deployed for single-species, real-time wildlife monitoring. Recent research has focused on lightweight and edge-optimized deep learning architectures that can deliver

high-accuracy detection while minimizing computational overhead, paving the way for practical applications in elephant conservation and human-elephant conflict (HEC) mitigation [3].

C. CNN-BASED METHOD FOR ELEPHANT DETECTION

The use of CNN-based methods has become increasingly popular in object recognition and image classification. Compared to traditional detection methods, CNN-based approaches require less manual intervention and demonstrate strong feature extraction capabilities, as well as good generalization. In recent years, many researchers have developed elephant detection models on CNN architectures. Visual detection-based methods are typically categorized into two main groups. The first is the one-stage detection model, such as the You Only Look Once (YOLO) series, Single Shot MultiBox Detector (SSD) [17], and RetinaNet [18]. The second is the two-stage detection model, represented by models such as Faster R-CNN [19], Cascade R-CNN and Mask R-CNN.

Chen et al. proposed YOLO-SAG, an enhanced version of the YOLOv8n model designed for wildlife detection [20]. The model integrates the Softplus activation function to improve training stability and accuracy, incorporates Adaptive Intra-Feature Interaction (AIFI) for better feature interaction across scales, and utilizes GSConv and VoV-GSCSP modules to reduce computational redundancy. The enhanced detection framework achieved 94.9% precision and reduced the computational complexity by 11.1%, outperforming YOLOv8n in wildlife detection tasks.

Roy et al. introduced WilDect-YOLO [10], a deep learning-based real-time detection model for endangered wildlife species. The system employs residual blocks in the CSPDarknet53 backbone and DenseNet blocks to enhance the spatial feature extraction and information preservation. Additionally, Spatial Pyramid Pooling (SPP) and a modified Path Aggregation Network (PANet) were used to improve the receptive field representation. On a custom dataset of endangered species, WilDect-YOLO achieved a mean average precision (mAP) of 96.89% and an F1-score of 97.87%, demonstrating superior performance in complex environments.

Gunasekara et al. developed a vision-based early warning system that leverages YOLOv3 to detect elephants and prevent train collisions in Sri Lanka [21]. YOLOv3's fully convolutional architecture, with 106 layers, enables the accurate detection of elephants at various scales. The system was trained on a custom dataset collected under various environmental conditions, achieving sufficient accuracy for reliable real-time deployment under diverse lighting scenarios.

Leonid et al. presented a deep learning-based approach using the YOLOv4 model for wildlife detection to mitigate human-wildlife conflicts [22]. Their system was trained on a dataset comprising over 10,000 images and achieved a detection accuracy of 94%, outperforming other models

due to YOLOv4's superior feature extraction and real-time performance capabilities.

Dave et al. proposed a real-time wild animal detection framework utilizing YOLOv8 to track and detect animals such as elephants, tigers, leopards, and bears from camera footage [23]. The dataset comprised 1619 annotated images, and the YOLOv8x model achieved a mAP of 94.3% with real-time detection at 20 FPS. Data augmentation techniques further enhance the model accuracy, demonstrating their effectiveness in real-world field applications for wildlife tracking.

Despite significant progress in deep learning-based elephant detection, existing models face several key challenges. Many YOLO-based wildlife detection systems are designed for multi-species classification, often resulting in suboptimal performance for species-specific applications, such as elephant detection. Computationally expensive architectures such as WilDect-YOLO [10] and Faster R-CNN achieve high detection accuracy but require powerful GPUs, making them unsuitable for real-time deployment on edge devices. While lightweight models like YOLOv8n offer faster inference speeds, they struggle with occlusions, scale variations, and high false-positive rates, particularly in dense forest environments. Unlike these multi-class object detection models, YOLO-FES adopts a specialized single-class approach explicitly tailored for elephants, allowing it to enhance detection accuracy and robustness while addressing the unique challenges of elephant detection in real-world HEC scenarios. By focusing exclusively on elephant intrusion detection, YOLO-FES reduces misclassification errors, improves precision, and maintains computational efficiency, making it a more practical and effective solution for conservation and conflict mitigation efforts.

In summary, the reviewed literature confirms significant progress in CNN-based wildlife detection, while also highlighting a recurring trade-off between detection accuracy and model complexity. Heavyweight models provide high precision but are impractical for scalable, real-time field deployment due to their reliance on powerful GPUs. Meanwhile, existing lightweight alternatives often compromise on performance, struggling with occlusions, scale variations, and false positives in challenging environments. This underscores the clear and pressing need for an optimized, lightweight architecture, such as the proposed YOLO-FES, that can deliver robust detection performance without incurring high computational overhead.

III. MATERIALS AND METHODS

This section first introduces the baseline YOLOv8n architecture, which serves as the foundational model for our proposed enhancements, as illustrated in Fig. 1. To address the specific challenges associated with elephant detection in real-world scenarios, we introduce YOLO-FES, an optimized and lightweight variant of YOLOv8n. The comprehensive architecture of YOLO-FES is depicted in Fig. 2, with the key modifications clearly highlighted in the enlarged and dotted

regions to emphasize the improvements made in feature extraction, attention mechanisms, and feature fusion.

A. YOLOv8 OVERVIEW

YOLOv8 is a real-time object detection algorithm developed by Ultralytics in January 2023. YOLOv8 delivers improved detection speeds and a more streamlined model compared to earlier YOLO versions, improving its efficiency in real-world applications. In addition, it predicts bounding boxes and class probabilities from complete images in a single pass, seamlessly combining speed, accuracy, and efficiency. The architecture consists of three core components: the backbone for feature extraction, the neck for feature fusion, and a detection head, that utilizes a decoupled, anchor-free approach to improve the bounding box alignment with objects.

The backbone of YOLOv8 is designed for efficient feature extraction by leveraging a combination of Convolutional (Conv2d), Batch Normalization (BN), and Sigmoid Linear Unit (SiLU). The Conv module is composed of convolutional layers combined with batch normalization and SiLU activation functions, which capture spatial hierarchies and stabilize training with improved gradient flow. Downsizing the kernel size of the first convolutional layer from 6×6 to 3×3 is one of the recent updates performed in YOLOv8, ensuring better precision and reduced computational cost. Moreover, C2f is inspired by the C3 module of YOLOv7 and the idea of ELAN, which proposes split operations and extra skip connections for gradient flow and feature reuse, while maintaining a lightweight architecture. The backbone also uses the SPPF module, which pools features from several receptive fields to enable the model to handle objects of different scales precisely. It serves as an innovative backbone design to broaden the robustness and adaptability of YOLOv8 extensively in general object detection tasks.

Meanwhile, the neck serves as a crucial intermediary between the backbone and the detection head, enhancing the model's ability to handle multi-scale object detection. It combines Conv and C2f modules with PANet [24] to refine the feature hierarchy. PANet facilitates the integration of fine-grained details from lower-level features with abstract patterns from higher-level features, thereby ensuring a seamless flow of multilevel contextual information. Compared to YOLOv5, YOLOv8 simplifies the neck by removing two convolutional layers before the upsampling process, improving efficiency. This design leverages both top-down and bottom-up pathways to merge deep and shallow semantic information, enabling robust detection of objects of varying sizes and complexities.

The head component of YOLOv8 has been significantly modified by dividing the classification and localization tasks, allowing each to be improved separately for better accuracy. This design guarantees thorough coverage of a range of object sizes by working across many detection layers, each of which maps to a different scale inside the feature pyramid. An anchor-free paradigm in decoupled head design

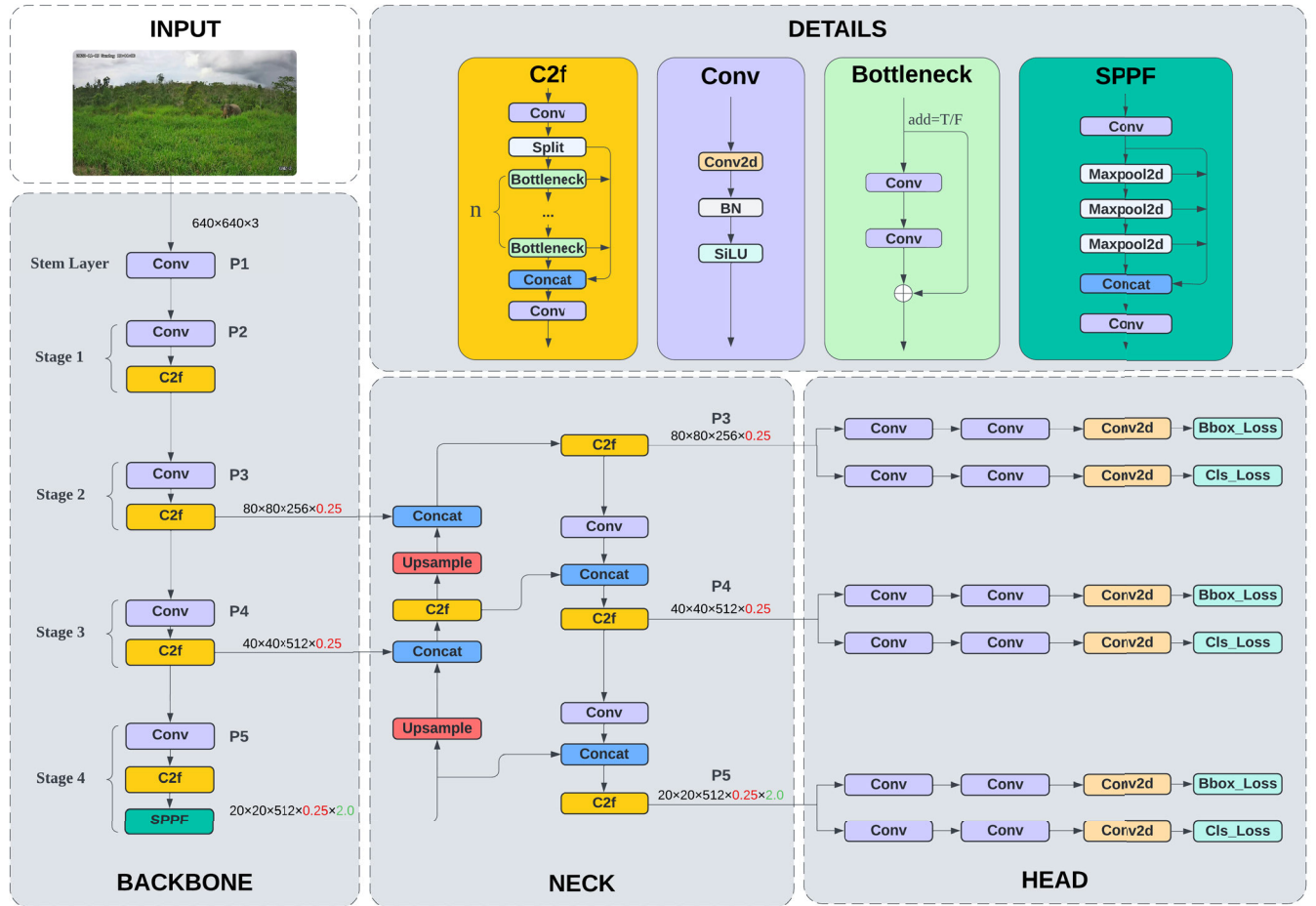


FIGURE 1. YOLOv8n network structure.

reduces dependency on anchor boxes, which traditionally offer preliminary estimations for object location and scale. YOLOv8 improves performance in real-time scenarios by separating classification from localization and implementing an anchor-free technique, which increases detection speed and produces more accurate bounding-box refinements.

B. IMPROVEMENT ON FEATURE EXTRACTION MODULE

1) C2F-FASTER MODULE

In YOLOv8, the C2f module replaced the C3 module from YOLOv5 to enhance high-quality feature extraction. To improve the efficiency of this module and reduce its parameter count, we propose C2f-Faster, a novel module that replaces the standard Bottleneck block in C2f [25] with the FasterNet block. C2f-Faster leverages the concept of Partial Convolution (PConv), which efficiently applies filters to only a subset of input channels while leaving the remaining channels unchanged. The structures of the FasterNet block and PConv are illustrated in Fig. 3.

Assuming that h and w denote the height and width of the input feature map, respectively, k represents the convolution kernel size, c indicates the number of input channels, and c_p denotes the channels involved in the convolution operation.

The FLOPs for PConv can be calculated as in the equation shown in (1).

$$\text{FLOPs}_{\text{PConv}} = h \times w \times k^2 \times c_p^2 \quad (1)$$

The FLOPs of PConv are reduced to just 1/16 of a standard convolution operation when implemented with a partial ratio of $r = \frac{c_p}{c} = 1/4$, significantly improving computational efficiency. Additionally, the memory access requirement of PConv can be expressed as shown in Equation (2).

$$\text{Memory}_{\text{PConv}} = h \times w \times 2c_p + k^2 \times c_p^2 \approx h \times w \times 2c_p \quad (2)$$

Since feature map access outweighs kernel weight access, this approximation is accurate. Compared to standard convolution operations, PConv effectively reduces both computational complexity and memory requirements, as evidenced by memory access being just 1/4 of regular convolution when $r = 1/4$.

To ensure the accuracy of vision tasks across various devices, the FasterNet Block, designed using PConv as its core construction operator, delivers enhanced operating speeds. The FasterNet Block has shortcut connections to reuse key features and consists of a PConv followed by two 1×1 convolution layers as the reverse residual blocks. The

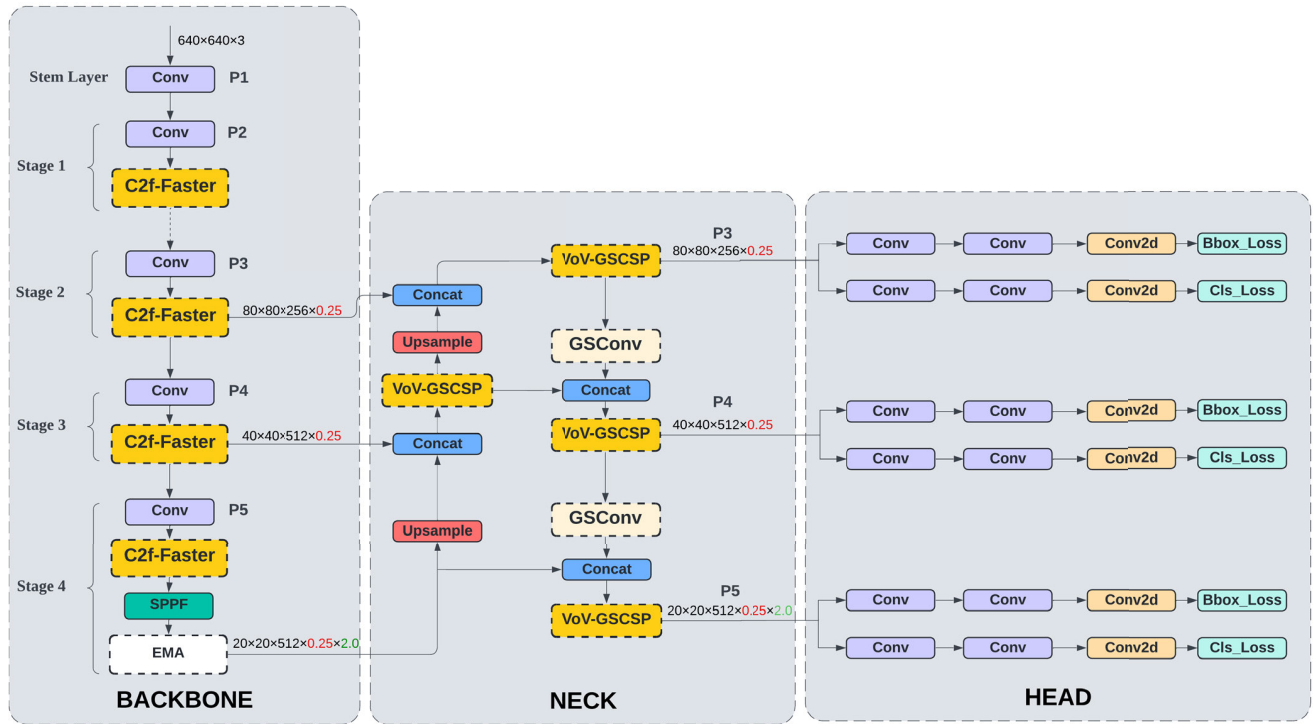


FIGURE 2. YOLO-FES network structure. Key architectural improvements are highlighted with enlarged and dotted box.

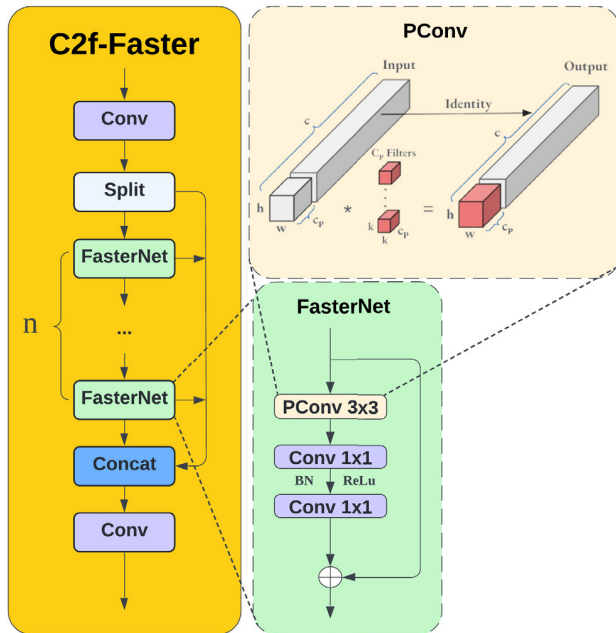


FIGURE 3. Structures of C2f-faster, fasterNet and PConv.

intermediate 1×1 convolution layer, which maintains feature diversity and achieves decreased delay, is utilized before the normalization and activation layers.

2) EMA MODULE

Inspired by the study of human vision, attention mechanisms offer an efficient way to improve a model's ability to extract important information in challenging environments. Deep

learning uses attention techniques to mimic how humans selectively focus on significant visual information while removing unimportant details. This allows neural networks to focus their computational resources on the most relevant features by prioritizing particular parts of the input data. This selective focus not only improves the model's ability to process complex data but also enhances performance and generalization.

By addressing the unique challenges of recognizing elephants in various environments, we incorporated the EMA [26] module, as shown in Fig. 4, into the backbone of YOLOv8 to enhance the model's feature extraction capability.

The module operates in three stages. Initially, for any input feature map $X \in \mathbb{R}^{C \times H \times W}$, EMA employs feature grouping by dividing it into G sub-features across the channel dimension and reshaping channels into batch dimensions to process semantic information at multiple scales efficiently.

In the second stage, EMA implements a parallel processing architecture with three routes: two in the 1×1 branch and one in the 3×3 branch. The 1×1 branch employs two parallel 1D global average pooling operations (horizontal and vertical) to encode channel attention along different spatial directions, as expressed in (3) and (4), respectively.

$$z_c^H = \frac{1}{W} \sum_{i=0}^{W-1} x_c(H, i) \quad (3)$$

$$z_c^W = \frac{1}{H} \sum_{j=0}^{H-1} x_c(j, W) \quad (4)$$

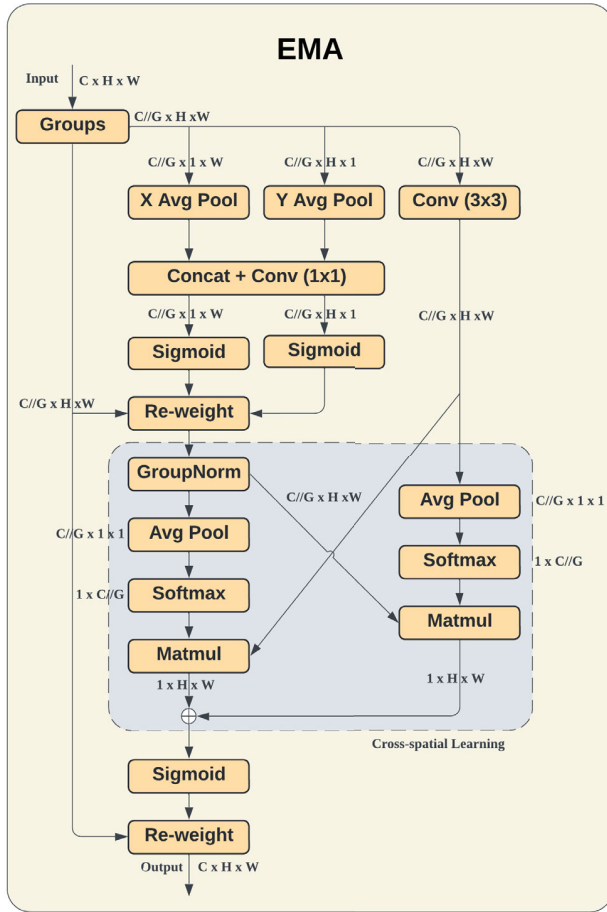


FIGURE 4. Structure of EMA.

While the 3×3 branch expands the feature space by capturing local cross-channel interactions, these encoded features are concatenated and share the same 1×1 convolution without dimension reduction. Finally, EMA introduces a cross-spatial learning technique that uses matrix dot-product operations and 2D global pooling to combine data across many spatial dimensions. This multi-scale approach enables the module to simultaneously capture both short and long-range dependencies while preserving precise positional information, ultimately producing more accurate pixel-level attention for feature maps.

Hence, incorporating the EMA module into YOLOv8n's backbone significantly enhances elephant detection by capturing both fine-grained details and global context. Through parallel 1×1 and 3×3 convolution branches and cross-spatial learning, EMA efficiently processes multi-scale features while preserving positional information. This enables YOLOv8n to accurately detect elephants under varying scales, partial occlusions, and diverse environmental conditions, all with minimal computational overhead.

C. IMPROVEMENT ON FEATURE FUSION MODULE

1) SLIM-NECK MODULE

In YOLOv8, the neck serves as a link between the head and the backbone networks, allowing the features to

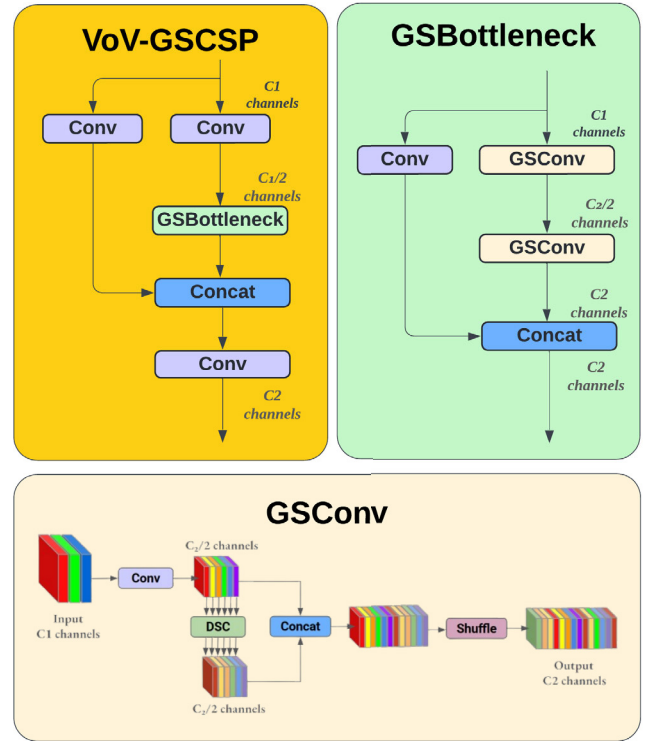


FIGURE 5. Structure of slim-neck.

be processed and fused. Although the C2f modules and Standard Convolutions (SC) boost accuracy, they also slow down computation and increase the number of parameters. To tackle these issues, researchers have explored alternatives such as Depthwise Separable Convolution (DSC) [27], which enhances efficiency and lowers model complexity. However, these techniques generally struggle to sustain the accuracy levels obtained by SC structures. Hence, in this study, we propose a novel GSConv by Slim-neck [28] to address the challenge.

Slim-neck architecture consists of three key components: GSConv, GSBottleneck, and VoV-GSCSP block as shown in Fig. 5.

GSConv adopts a hybrid strategy that combines SC for rich feature extraction and DSC for improved efficiency. By maintaining inter-channel connections via a dual-path approach, it mitigates the loss of semantic information that occurs during spatial compression in CNNs. The SC and DSC outputs are concatenated and shuffled to efficiently combine features, thereby improving feature representation and computational efficiency. GSConv preserves the same learning ability and feature richness at approximately half the computational cost of SC [28].

Additionally, the VoV-GSCSP block, which enables cross-level partial networks through a one-time aggregation technique, is incorporated into the Slim-neck structure. This architecture minimizes computing costs while ensuring the efficient fusion of multi-scale information. When combined, GSConv, GSBottleneck, and VoV-GSCSP significantly reduce model complexity and parameter counts,

making Slim-neck the perfect choice for lightweight and compatible edge device deployments. Slim-neck enhances YOLOv8's efficiency by simplifying the network while maintaining high detection accuracy, particularly in resource-constrained scenarios, such as real-time elephant intrusion detection on edge devices.

IV. EXPERIMENTS

This section outlines the experimental setup, dataset specifications, evaluation metrics, and comparative analysis employed to evaluate the performance of YOLO-FES. The experiments are designed to validate the effectiveness of the proposed model in terms of detection accuracy, computational efficiency, and real-time feasibility. We first outline the hardware and software environments used for model training and inference. Next, we describe the dataset preprocessing pipeline and augmentation techniques to ensure robust generalization across diverse environmental conditions. Finally, we define the performance metrics and benchmarking strategy, comparing YOLO-FES against state-of-the-art models, including YOLOv8 variants, Faster R-CNN, and SSD, to highlight its improvements in precision, inference speed, and deployment efficiency.

A. DATASET

The elephant image data for this study were acquired through the online platform, Roboflow [29], and supplemented with elephant images from MSCOCO2017 [30]. In total, the dataset comprised 4,531 images with 6,249 annotations, which were then divided into training, testing, and validation splits. These images in the dataset were collected from various sources, including trap cameras, CCTV cameras, televisions, drones, and mobile phones. This diversity in image acquisition methods contributes significantly to the robustness and variability of the dataset, thereby enhancing its applicability across multiple contexts of elephant detection. Due to the varied sources, image sizes ranged from tiny, small, medium, large, and jumbo, with 100px, 224px, 512px, 1024px, and 11848px, respectively. Hence, the median image ratio for this dataset was 640×480 pixels, with an average image size of 0.31 megapixels.

To ensure the dataset reflects real-world scenarios, it was curated to include diverse conditions commonly encountered in HEC zones, such as dense vegetation, occlusions, and varying lighting conditions. This focus on real-world applicability aligns with the single-class objective of YOLO-FES, allowing the model to be explicitly fine-tuned for detecting elephants in challenging environments.

Fig. 6 illustrates the detailed information of the dataset. The number of elephant instances in the dataset is shown in Fig. 6a. The elephant bounding box sizes are displayed in Fig. 6b, which also displays the range of the bounding box dimensions. The distribution of the centroid coordinates of the labeled box is shown in Fig. 6c, indicating that most center points are concentrated in the central area of the pictures. A scatter plot of the width-to-height ratio of the labeled boxes

is shown in Fig. 6d, indicating that the dataset mainly consists of small to medium-sized targets with different aspect ratios.

B. EXPERIMENT SETUP

The training was performed on a local machine running Ubuntu 22.04.4 LTS with an Intel Core i7-12700 CPU and a twin NVIDIA GeForce RTX 3060 GPU. The environment utilized CUDA 12.1 for GPU acceleration alongside PyTorch 2.3.1 and Torchvision 0.18.1, with Python 3.10.12 serving as the programming language, as summarized in Table 1.

TABLE 1. Experiment environment.

Model	Experiment environment
OS	Ubuntu 22.04.4 LTS
CPU	i7-12700
GPU	RTX3060
CUDA	12.1
Python	3.10.12
Pytorch	2.3.1
Torchvision	0.18.1

Meanwhile, for edge deployment, testing was conducted on three edge devices: Raspberry Pi 4B, Industrial Mini PC, and Jetson Orin Nano. The specifications for each hardware are presented in Tables 2, 3, and 4, respectively.

TABLE 2. Experiment environment for raspberry Pi 4B.

Component	Specification
OS	Raspberry Pi OS (64-bit)
CPU	Broadcom BCM2711 (Quad-core Cortex-A72)
RAM	4GB
Python	3.10.12
Pytorch	2.3.1
Torchvision	0.18.1

TABLE 3. Experiment environment for industrial mini PC.

Component	Specification
OS	Ubuntu 22.04.5 LTS
CPU	Intel Core i5-8260U
RAM	8GB
Python	3.10.12
Pytorch	2.0.1
Torchvision	0.15.2

TABLE 4. Experiment environment for jetson orin nano.

Component	Specification
OS	JetPack 5.1.2 (Ubuntu 20.04)
CPU	6-core Arm Cortex-A78AE
GPU	512-core NVIDIA Ampere
RAM	4GB
Python	3.8.10
Pytorch	2.1.0
Torchvision	0.16.2

Each model underwent 200 training epochs with a batch size of 16. The Stochastic Gradient Descent (SGD) optimizer

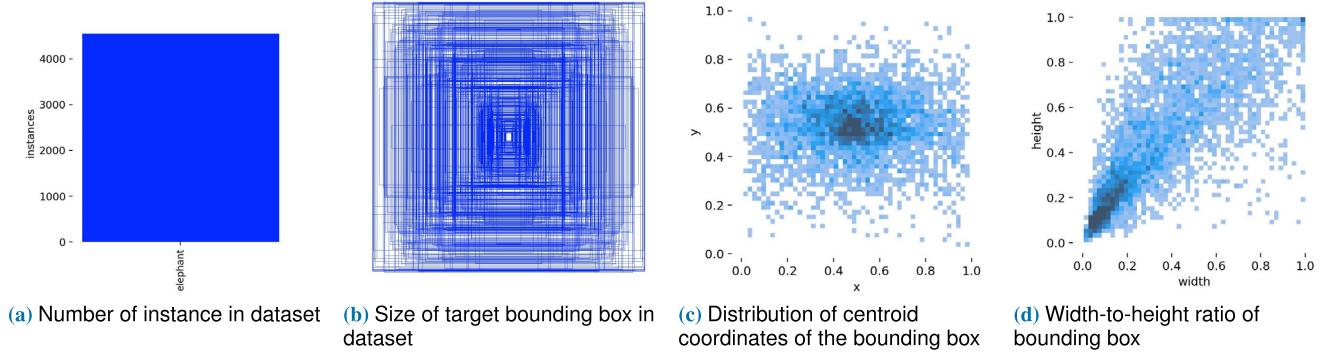


FIGURE 6. Dataset information.

was utilized, with momentum, weight decay, and initial learning rates of 0.937, 0.0005, and 0.01, respectively. None of the pre-trained models was used in this study. The hyperparameter configurations for the model training are summarized in Table 5.

TABLE 5. Experiment parameters and configurations.

Parameter	Configuration
Input Size	640
Batch Size	16
Epochs	200
Optimizer	SGD
Momentum	0.937
Weight Decay	0.0005
Initial Learning Rate	0.01

C. EVALUATION METRICS

In the objective evaluation of the elephant detection models, a range of comprehensive evaluation metrics was used to capture various aspects of model performance. This study focused not only on the accuracy of the models, but also on their lightweight requirements for deployment in edge computing devices. Hence, the average precision (AP), mean average precision (mAP), floating-point operations (FLOPs), parameters (Params), and model size were selected as the key evaluation indices.

AP represents the area under the precision-recall curve, with values closer to 1 indicating better model performance. The formula is shown in (5).

$$AP = \sum_n (R_n - R_{n-1}) P_n \quad (5)$$

mAP is the average of AP of all sample categories, which is the most commonly used evaluation metric in object detection and intuitively reflects the performance of the current model, as shown in (6).

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i \quad (6)$$

FLOPs can reflect the computational complexity of the model, which is calculated as shown in (7)

$$FLOPs = 2 \times (k_h \times k_w \times C_{in} \times H_{out} \times W_{out} \times C_{out}) \quad (7)$$

The lightweight of a model can be determined by the number of Params, which is calculated as shown in (8)

$$Params = (k_w \times k_h \times C_{in} + 1) \times C_{out} \quad (8)$$

V. RESULTS AND DISCUSSION

This section presents the quantitative and qualitative evaluation of the proposed YOLO-FES model, assessing its detection accuracy, computational efficiency, and real-time feasibility. The performance of YOLO-FES is compared against baseline models, including YOLOv8n, WilDect-YOLO, and Faster R-CNN, to highlight its advantages in terms of mean average precision (mAP), inference speed, model size, and computational complexity. Additionally, edge deployment experiments are conducted to validate the model's real-time detection capabilities on resource-constrained devices.

A. ABLATION EXPERIMENTS

Ablation experiments were conducted on YOLO-FES to evaluate the impact of each improvement strategy proposed in this study, and the results are displayed in Table 6.

From Table 6, by replacing the C2f module in YOLOv8n with the C2f-Faster module (model variant a), we observed a 23.3%, 22.2%, and 22.5% decrease in the number of Params, FLOPs, and model size, respectively. This improvement significantly reduces the computational cost and size of the model, thereby enhancing feature extraction efficiency. However, the mAP decreased slightly to 56.6%. The FasterNet block, integrated into the standard C2f module, leverages the concept of PConv, filters are applied to only a small number of input channels, proving to be both fast and efficient.

The integration of Slim-neck (model variant b) resulted in a slight decrease in mAP of 0.7%, while achieving a significant reduction in FLOPs of 8.6% and a moderate decrease in Params of 4.0%, highlighting its efficiency. Although Slim-neck integration aims to simplify the neck structure of

TABLE 6. Detection results after the introduction of different improvement strategies.

Model	C2F-Faster	Slim-neck	EMA	mAP@0.5-0.95(%)	Params (M)	FLOPs (G)	Model Size (MB)	Speed (ms)
YOLOv8n	-	-	-	57.2	3.0	8.1	6.2	1.3
a	✓	-	-	56.6	2.3	6.3	4.8	1.2
b	-	✓	-	56.8	2.88	7.4	6.1	1.4
c	-	-	✓	58.1	3.0	8.1	6.2	1.1
d	✓	✓	-	57.7	2.43	6.2	5.1	1.0
e	-	✓	✓	57.8	2.79	7.3	5.9	1.1
f	✓	-	✓	56.2	2.3	6.3	4.8	1.0
YOLO-FES	✓	✓	✓	58.4	2.43	6.2	5.2	1.1

the network, this implies that additional optimization may be required to better balance the computational efficiency and performance. Nevertheless, Slim-neck's ability to maintain competitive accuracy while reducing architectural complexity through its innovative GSConv-based design demonstrates its potential as a promising approach for efficient feature fusion in object detection networks.

Subsequently, when EMA (model variant c) was implemented, we observed a notable improvement in mAP to 58.1%, while maintaining the same parameter count and model size. This effectiveness results from EMA's novel cross-spatial learning technique, which reduces computational costs while improving feature representation by efficiently capturing both short and long-range dependencies without requiring channel dimensionality reduction.

The combination of C2f-Faster and Slim-neck (model variant d) shows promising results, increasing the mAP to 57.7% while reducing the parameter count by 19.0% to 2.43M, the model size by 17.74% to 5.1MB, and the GFLOPs by 23.46% to 6.2G. This synergistic effect implies that these modifications complement each other effectively, achieving a strong balance between efficiency and precision.

Further experiments combining different strategies (variants e and f) show varied results; however, the final YOLO-FES architecture, which incorporates all three improvements (C2F-Faster, Slim-neck, and EMA), achieves the best overall performance, with a mAP of 58.4%, marking a 2.1% increase over the baseline. YOLO-FES achieves better efficiency by achieving reductions in Params, FLOPs, and model size by 19%, 23.5% and 16.1% respectively. These improvements demonstrate that the combined optimization strategies effectively enhanced both the model's accuracy and its resource efficiency.

B. COMPARISON EXPERIMENTS

To demonstrate the advantages of the YOLO-FES model, we conducted a performance comparison with previous state-of-the-art models. The models selected for this comparative analysis provide a robust and comprehensive benchmark. This selection includes classic two-stage detectors (Faster R-CNN), influential one-stage detectors (SSD), and a wide array of modern, lightweight YOLO variants (YOLOv3-tiny through YOLOv8s). By evaluating YOLO-FES against these diverse architectures, we can provide a thorough assessment

of its performance relative to the broader state-of-the-art in object detection. The key metrics for evaluating each model in this experiment were mAP@0.5, Params, FLOPs, and Model Size. The experimental results are listed in Table 7.

As shown in Table 7, YOLO-FES achieved the highest detection accuracy of 88.8% and 58.4% for mAP@50 and mAP@50-95 respectively. These values outperform all other models in the comparison group, including YOLOv8n and YOLOv8s, which have mAP@0.5-0.95 values of 57.2%. Traditional models such as Faster R-CNN with mAP@0.5-0.95 of 48.5% and SSD with 46.2%.

In terms of efficiency, YOLO-FES demonstrated exceptional computational and memory performance. With just 2.43M Params, 6.2G FLOPs, and a model size of 5.2 MB, it is the most lightweight model in the comparison. YOLO-FES reduces parameters by 40.4% compared to YOLOv8n with 3.0M and 91.6% compared to Faster R-CNN with 41.2M. Its FLOPs are 25.8% lower than YOLOv8n with 8.1G and significantly smaller than SSD with 88.6G. These reductions in complexity make YOLO-FES an ideal choice for deployment in resource-constrained environments, particularly on edge devices.

These comprehensive results demonstrate that YOLO-FES successfully achieves its design objectives, offering a balanced solution that enhances detection accuracy while substantially reducing computational overhead and model complexity. The ability of the model to maintain high performance while minimizing resource requirements makes it particularly valuable for real-world elephant intrusion detection applications, where both accuracy and efficiency are crucial.

C. ATTENTION MECHANISM COMPARISON

To evaluate the effectiveness of different attention mechanisms in elephant intrusion detection, we compared the performance of YOLOv8 integrated with various attention modules, including Coordinate Attention (CA) [33], Efficient Channel Attention (ECA) [34], Simple Attention Module (SimAM) [35], and EMA [26]. As shown in Table 8, the EMA model outperforms the others, with the highest mAP@0.5 of 58.1%, while achieving the fastest processing speed of 1.1 ms. The SimAM model achieves an mAP of 57.7%, offering a balance between performance and speed. The CA model, despite reaching an mAP of 56.5%, has a higher

TABLE 7. Performance of different models on the elephant dataset.

Model	mAP @0.5 (%)	mAP @0.5-0.95 (%)	Params (M)	FLOPs (G)	Model Size (MB)	Speed (ms)
Faster R-CNN	81.3	48.5	41.2	181.5	98.4	85.6
SSD	79.8	46.2	27.5	88.6	71.3	73.8
YOLOv3-tiny [31]	83.7	52.0	12.1	18.9	24.4	1.4
YOLOv5n [8]	88.3	56.5	2.5	7.1	5.3	1.6
YOLOv6n [32]	87.3	56.6	4.23	11.8	8.7	1.2
YOLOv7-tiny [9]	86.9	54.5	6.0	13.0	12.3	2.0
YOLOv8n	87.3	57.2	3.0	8.1	6.2	1.3
YOLOv8s	86.4	57.2	11.1	28.4	22.5	2.3
YOLO-FES	88.8	58.4	2.43	6.2	5.2	1.1



FIGURE 7. EigenGrad-CAM visualisation results.

TABLE 8. Comparison with other attention mechanisms.

Model	mAP@0.5 (%)	Params (M)	FLOPs (G)	Speed (ms)
CA	56.5	3.03	8.1	1.8
ECA	56.4	3	8.1	1.6
SimAM	57.7	3	8.17	1.6
EMA	58.1	3	8.1	1.1

parameter count of 3.03M and a slower speed of 1.8 ms. Similarly, the ECA model achieves an mAP of 56.4% with comparable computational complexity but slightly slower processing speed compared to EMA. These findings highlight EMA’s superior performance and efficiency for elephant intrusion detection.

D. VISUAL ANALYSIS

EigenGrad-CAM is a visualization method that generates heatmaps to aid in understanding neural network

predictions. Furthermore, EigenGrad-CAM utilizes the principal components of the activation maps, multiplied by gradients, to emphasize the most important features that influence the model’s decisions [36], in contrast to conventional Class Activation Mapping (CAM) techniques. This approach offers clearer interpretability and a more focused visualization of the relevant regions in the input data. In this study, we employed EigenGrad-CAM to generate attention heatmaps for the base model and YOLO-FES.

For our test cases, we chose pictures of elephants with intricate and varied backgrounds. These images included scenes with one or more elephants and a range of distances, from close-up to distant locations. These backgrounds include dense vegetation, such as trees and grasses, which introduce significant complexity. The performance of the target detection model can be tested and affected by such complex scenarios. The comparison is shown in Fig. 7.

According to Fig. 7, although YOLOv8n generally performs well in detection tasks, its focus is occasionally diverted to non-target regions due to significant background interference, and it fails to focus on elephants at close range correctly. By contrast, YOLO-FES exhibits greater robustness in target detection and maintains a more intense focus on elephants, even in complex settings.

To evaluate the effectiveness of the YOLO-FES, we performed a visual comparison with the baseline model. This comparison highlights the performance of each model in identifying elephants within complex backgrounds, as shown in Fig. 8

According to the Fig. 8, we observed that YOLO-FES performed better than the baseline model. The results demonstrate that YOLO-FES outperforms the baseline model, yielding higher confidence scores for each bounding box and thereby showcasing its strong capability to detect and localize elephants even in visually intricate environments.

Our model also demonstrated excellent performance, particularly in scenes with varied and dense backgrounds. Its reliability and accuracy in distinguishing targets from non-target regions are demonstrated by its continuously higher confidence scores. This superior capability highlights the advancements in YOLO-FES's feature extraction and fusion modules, making it a reliable and efficient model for elephant intrusion detection.

E. MODEL DEPLOYMENT ON EDGE DEVICES

To evaluate the practicality and efficiency of YOLO-FES in real-world scenarios, we deployed the model on three edge devices: Raspberry Pi 4B, an Industrial Mini PC, and Jetson Orin Nano. Each device represents a different class of edge computing hardware, allowing us to assess the performance of the model across a range of resource-constrained environments. The deployment process involved converting the model into formats optimized for each device's hardware architecture, ensuring efficient inference while maintaining a high detection accuracy.

- 1) **Raspberry Pi 4B:** The Raspberry Pi 4B is a low-cost, energy-efficient single-board computer widely used in edge computing applications. To optimize YOLO-FES for deployment on the Raspberry Pi, we converted the model to NCNN format [37], a high-performance neural network inference framework optimized for mobile and edge devices. NCNN leverages the ARM CPU architecture of the Raspberry Pi, enabling efficient inference with minimal memory and power consumption.
- 2) **Industrial Mini PC:** The Industrial Mini PC represents a mid-range edge computing device equipped with an Intel CPU. To optimize YOLO-FES for this hardware, we utilized the OpenVINO toolkit [38], which is specifically designed to accelerate deep learning inference on Intel processors. OpenVINO optimizes the model by converting it into an Intermediate Representation (IR)

format and leveraging hardware-specific optimizations such as Intel DL Boost and AVX-512 instructions.

- 3) **Nvidia Jetson Orin Nano:** The Jetson Orin Nano is a compact, GPU-accelerated edge computing device designed for AI and DL applications. To optimize YOLO-FES for the Jetson Orin Nano, we converted the model into TensorRT format [39], which leverages the device's NVIDIA GPU for accelerated inference. TensorRT optimizes the model by applying techniques such as layer fusion, precision calibration, and dynamic tensor memory management.

The results of deployment are shown in Table 9.

TABLE 9. Deployment results on different edge devices.

Device	Framework	Speed (ms)
Raspberry Pi 4B	Pytorch	855.4
	NCNN	254.2
Industrial Mini PC	Pytorch	63.8
	OpenVINO	52.3
Jetson Orin Nano	Pytorch	30.5
	TensorRT	24.7

As shown in Table 9, the results of the deployment in various edge devices show notable differences in the speed of inference, which is essential for the quick detection of elephant intrusions in HEC zones. Particularly when using the PyTorch framework, the Raspberry Pi 4B exhibits comparatively high latency with 855.4 ms, despite being cost-effective for edge deployments. The latency is reduced to 254.2 ms when the NCNN framework is used, highlighting the significance of the framework optimization for field deployments with limited resources.

The Industrial Mini PC performs better with PyTorch, achieving a latency of 63.8 ms. In comparison, OpenVINO further reduces the latency to 52.3 ms, demonstrating the advantages of hardware acceleration for faster elephant detection. Notably, the Jetson Orin Nano achieves the lowest latency, with 30.5 ms using PyTorch and 24.7 ms using TensorRT. This makes it ideal for real-time elephant monitoring, where immediate recognition is crucial to preventing human-elephant conflicts.

In conclusion, the choice of hardware and framework has a significant impact on the effectiveness of elephant intrusion detection systems. For critical areas that require real-time monitoring, high-performance devices such as the Jetson Orin Nano, paired with optimized frameworks like TensorRT, are recommended. For widespread deployment across larger agricultural areas where cost is a constraint, leveraging optimized frameworks such as NCNN on devices such as the Raspberry Pi 4B can provide a viable balance between performance and cost-effective elephant detection.

F. LIMITATIONS OF THE STUDY

This study has several limitations that should be taken into consideration. First, the model was designed exclusively for elephant detection. Its specificity remains untested against



FIGURE 8. Detection results between YOLO-FES and baseline model.

other large, visually similar mammals, such as rhinoceroses, which could lead to false positives in diverse ecosystems. Second, although the training dataset is diverse, its representation of extreme environmental conditions, such as severe weather, dense fog, or nighttime scenarios, is limited. Consequently, the model's robustness may be reduced in these challenging conditions. The influence of specific factors, such as target pose and partial occlusion, also warrants further investigation. Finally, there is a practical trade-off in edge deployment. While YOLO-FES performs efficiently on capable devices like the Jetson Orin Nano, its inference speed on lower-cost hardware, such as the Raspberry Pi 4B (254.2 ms), may not be sufficient for all real-time monitoring applications. Future work will address these limitations by expanding the dataset to multiple species, testing robustness under varied environmental conditions, and exploring optimization techniques such as pruning and quantization for broader deployment.

VI. CONCLUSION

This study presents YOLO-FES, a lightweight object detection model specifically designed for efficient and precise elephant intrusion detection. Key enhancements include replacing the bottleneck in the C2f module with the novel FasterNet block, integrating EMA into the backbone, and utilizing the Slim-neck module powered by GSConv to improve feature fusion efficiency. These advancements enable the model to strike a balance between high detection accuracy and low computational overhead.

Meanwhile, the dataset used for training was gathered from various sources, including trap cameras, CCTV cameras,

drones, mobile phones, and television, and further enriched with elephant images from the MSCOCO2017 dataset.

Moreover, the results of our experiments show that our proposed model outperforms previous state-of-the-art models. In particular, it reduces the parameters, FLOPs, and model size by 19%, 23.5%, and 16.1%, respectively, while outperforming YOLOv8n in mAP@0.5 by 1.5% and mAP@0.5-0.95 by 1.2%.

To assess the real-world applicability of YOLO-FES, we deployed the model on different edge devices, demonstrating significant variations in inference speed depending on hardware capabilities and software frameworks. The Jetson Orin Nano achieved the best performance, with TensorRT reducing inference time to 24.7 ms, making it ideal for real-time applications. The Industrial Mini PC also provided competitive results, with OpenVINO optimizing inference to 52.3 ms. Meanwhile, the Raspberry Pi 4B, although cost-effective, exhibited higher latency, with PyTorch reaching 855.4 ms; however, NCNN improved performance to 254.2 ms. These findings highlight the importance of selecting both the right hardware and optimized inference frameworks to achieve real-time monitoring in resource-constrained environments.

Ultimately, the lightweight design of YOLO-FES makes it suitable for deployment on edge devices, enabling real-time monitoring in challenging deployment scenarios. By effectively balancing detection accuracy and computational efficiency, YOLO-FES proves to be a highly effective solution for real-time elephant intrusion detection.

Building on this work, future research will focus on two key areas. First, the dataset will be expanded to include other

wildlife species, thereby enhancing the model's specificity and enabling broader application in diverse ecosystems. Second, we will explore advanced optimization techniques, such as quantization and pruning, to further reduce the model's computational footprint and enable effective deployment on an even wider range of low-cost edge devices.

ACKNOWLEDGMENT

The authors sincerely acknowledge the use of AI tools, which contributed to editing and improving the readability of this article. However, the content, ideas, and conclusion presented are entirely the work of the authors, reflecting their original research and insights.

REFERENCES

- [1] J. A. de la Torre, E. P. Wong, A. M. Lechner, N. Zulaikha, A. Zawawi, P. Abdul-Patah, S. Saaban, B. Goossens, and A. Campos-Arceiz, "There will be conflict-agricultural landscapes are prime, rather than marginal, habitats for Asian elephants," *Animal Conservation*, vol. 24, no. 5, pp. 720–732, Oct. 2021.
- [2] L. Brickson, L. Zhang, F. Vollrath, I. Douglas-Hamilton, and A. J. Titus, "Elephants and algorithms: A review of the current and future role of AI in elephant monitoring," *J. Roy. Soc. Interface*, vol. 20, no. 208, Nov. 2023, Art. no. 20230367.
- [3] S. Gupta, N. Mohan, P. Nayak, K. C. Nagaraju, and M. Karanam, "Deep vision-based surveillance system to prevent train–elephant collisions," *Soft Comput.*, vol. 26, no. 8, pp. 4005–4018, Apr. 2022.
- [4] Y. Y. Loo, N. R. Avicena, N. Thong, A. Haque, Y. T. Nhlabatsi, S. Y. A. Abakar, K. H. Ng, and E. P. Wong, "WildTechAlert: Deep learning models for real-time detection of elephant presence using bioacoustics in an early warning system to support human–elephant coexistence," in *Proc. Adv. Brain Inspired Cognit. Syst.*, Singapore, J. Ren, A. Hussain, I. Y. Liao, R. Chen, K. Huang, H. Zhao, X. Liu, P. Ma, and T. Maul, Eds., Cham, Switzerland: Springer, 2024, pp. 385–399.
- [5] K. S. P. Premarathna, R. M. K. T. Rathnayaka, and J. Charles, "An elephant detection system to prevent human–elephant conflict and tracking of elephant using deep learning," in *Proc. 5th Int. Conf. Inf. Technol. Res. (ICITR)*, Dec. 2020, pp. 1–6.
- [6] M. I. M. Jamaluddin, K. Z. Abidin, S. M. Nor, A. Shukor, A. I. Zainuddin, R. Illias, and M. S. Mansor, "Asian elephants involved in conflicts exhibit similar habitat use but travel farther than non-conflict individuals," *Global Ecol. Conserv.*, vol. 55, Nov. 2024, Art. no. e03228.
- [7] S. D. Silva, T. Wu, P. J. Nyhus, A. Thieme, A. Weaver, J. M. Johnson, J. Wadey, A. M. Moßbrucker, T. T. Vu, T. Neang, B. S. Chen, M. Songer, and P. Leimgruber, "The past, present and future of elephant landscapes in Asia," *bioRxiv*, Apr. 2020.
- [8] G. Jocher, "Ultralytics YOLOv5," Ultralytics, Ballenger Creek, MD, USA, 2020. [Online]. Available: <https://docs.ultralytics.com/models/yolov5/#citations-and-acknowledgments>
- [9] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," 2022, *arXiv:2207.02696*.
- [10] A. M. Roy, J. Bhaduri, T. Kumar, and K. Raj, "WilDect-YOLO: An efficient and robust computer vision-based accurate object localization model for automated endangered wildlife detection," *Ecological Informat.*, vol. 75, Jul. 2023, Art. no. 101919.
- [11] S. Schneider, S. Greenberg, G. W. Taylor, and S. C. Kremer, "Three critical factors affecting automated image species recognition performance for camera traps," *Ecol. Evol.*, vol. 10, no. 7, pp. 3503–3517, Apr. 2020.
- [12] S. Schlossberg, M. J. Chase, and C. R. Griffin, "Testing the accuracy of aerial surveys for large mammals: An experiment with African savanna elephants (*Loxodonta Africana*)," *PLoS ONE*, vol. 11, no. 10, Oct. 2016, Art. no. e0164904.
- [13] M. D. Koneff, J. A. Royle, M. C. Otto, J. S. Wortham, and J. K. Bidwell, "A double-observer method to estimate detection rate during aerial waterfowl surveys," *J. Wildlife Manage.*, vol. 72, no. 7, pp. 1641–1649, Sep. 2008.
- [14] M. E. Deowan, A. K. Nuhel, I. Haider, M. E. A. Al Mahdi Tusher, M. M. Sazid, and M. N. M. Bhuiyan, "A warning and detection system for elephant intrusions utilizing signal processing and IoT," in *Proc. 4th Int. Conf. Electr., Comput. Telecommun. Eng. (ICECTE)*, Dec. 2022, pp. 1–4.
- [15] M. Zeppelzauer, "Automated detection of elephants in wildlife video," *EURASIP J. Image Video Process.*, vol. 2013, no. 1, p. 46, Aug. 2013.
- [16] V. Palanisamy and N. Ratnarajah, "Detection of wildlife animals using deep learning approaches: A systematic review," in *Proc. 21st Int. Conf. Adv. ICT Emerg. Regions (ICTer)*, Dec. 2021, pp. 153–158.
- [17] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot MultiBox detector," 2015, *arXiv:1512.02325*.
- [18] T. Lin, P. Goyal, R. B. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," 2017, *arXiv:1708.02002*.
- [19] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017.
- [20] L. Chen, G. Li, S. Zhang, W. Mao, and M. Zhang, "YOLO-SAG: An improved wildlife object detection algorithm based on YOLOv8n," *Ecol. Informat.*, vol. 83, Nov. 2024, Art. no. 102791.
- [21] S. Gunasekara, M. Jayasuriya, N. Harischandra, L. Samaranyake, and G. Dissanayake, "A convolutional neural network based early warning system to prevent elephant–train collisions," in *Proc. IEEE 16th Int. Conf. Ind. Inf. Syst. (ICIIS)*, Dec. 2021, pp. 271–276.
- [22] T. T. Leonid, H. Kanna, V. J. C. Christy, A. S. Hamritha, and C. Lokesh, "Human wildlife conflict mitigation using YOLO algorithm," in *Proc. 8th Int. Conf. Sci. Technol. Eng. Math. (ICONSTEM)*, Apr. 2023, pp. 1–7.
- [23] B. Dave, M. Mori, A. Bathani, and P. Goel, "Wild animal detection using YOLOv8," *Proc. Comput. Sci.*, vol. 230, pp. 100–111, Jan. 2023.
- [24] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path aggregation network for instance segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 8759–8768.
- [25] J. Chen, S.-H. Kao, H. He, W. Zhuo, S. Wen, C.-H. Lee, and S.-H.-G. Chan, "Run, Don't walk: Chasing higher FLOPS for faster neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 12021–12031.
- [26] D. Ouyang, S. He, G. Zhang, M. Luo, H. Guo, J. Zhan, and Z. Huang, "Efficient multi-scale attention module with cross-spatial learning," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Greece, Jun. 2023, pp. 1–5.
- [27] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1800–1807.
- [28] H. Li, J. Li, H. Wei, Z. Liu, Z. Zhan, and Q. Ren, "Slim-neck by GSConv: A lightweight-design for real-time detector architectures," *J. Real-Time Image Process.*, vol. 21, no. 3, p. 62, Jun. 2024.
- [29] (Oct. 2024). *Elephant Dataset*. [Online]. Available: <https://universe.roboflow.com/sandb0xgajah/elephant-drcwi>
- [30] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár, "Microsoft COCO: Common objects in context," 2014, *arXiv:1405.0312*.
- [31] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," 2018, *arXiv:1804.02767*.
- [32] C. Li, L. Li, Y. Geng, H. Jiang, M. Cheng, B. Zhang, Z. Ke, X. Xu, and X. Chu, "YOLOv6 v3.0: A full-scale reloading," Meituan, Beijing, China, Tech. Rep., 2023. [Online]. Available: <https://docs.ultralytics.com/models/yolov6/#citations-and-acknowledgments>
- [33] Q. Hou, D. Zhou, and J. Feng, "Coordinate attention for efficient mobile network design," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, TN, TN, USA, Jun. 2021, pp. 13708–13717.
- [34] Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, and Q. Hu, "ECA-Net: Efficient channel attention for deep convolutional neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Seattle, WA, USA, Jun. 2020, pp. 11531–11539.
- [35] L. Yang, R.-Y. Zhang, L. Li, and X. Xie, "Simam: A simple, parameter-free attention module for convolutional neural networks," in *Proc. 38th Int. Conf. Mach. Learn.*, vol. 139, pp. 11863–11874.
- [36] J. Gildenblat. (2021). *Pytorch Library for Cam Methods*. [Online]. Available: <https://github.com/jacobgill/pytorch-grad-cam>
- [37] H. Ni, "NCNN," Tencent, Shenzhen, China, Tech. Rep., Jun. 2017. [Online]. Available: <https://github.com/Tencent/ncnn/blob/master/CITATION.cff>
- [38] *OpenVINOT Toolkit*. Accessed: Nov. 30, 2024. [Online]. Available: <https://github.com/openvinotoolkit/openvino>
- [39] *TensorRT*. Accessed: Nov. 30, 2024. [Online]. Available: <https://github.com/NVIDIA/TensorRT>



MUHAMMAD DANIAL MOHAMAD RIZWAN is currently pursuing the master's (by research) degree in soft computing and intelligent systems with the Universiti Malaysia Pahang Al-Sultan Abdullah (UMPSA), Pekan, Pahang, Malaysia. He has been a Research Assistant with the Soft Computing and Intelligent Systems Laboratory, UMPSA, since 2023. His research interests include machine learning, deep learning, network security, and computer vision techniques for real world applications.



MD. AKBAR HOSSAIN (Senior Member, IEEE) received the M.Sc. degree in telecommunication engineering from the University of Trento, Italy, and the Ph.D. degree from the School of Engineering, Computer and Mathematical Sciences, Auckland University of Technology, New Zealand. He is currently a Senior Lecturer with the School of Computing, Eastern Institute of Technology (EIT). After completing master's studies, he received the prestigious Marie Curie Early-Stage Research

Fellowship to work as a Researcher in European Union (EU) funded projects in Germany and Greece. His area of research interests include wireless networks, wireless sensor networks, cybersecurity, artificial intelligence, and cognitive radio networks. He is an Active Reviewer of many reputed journals, such as JNCA, *Ad-Hoc Networks* (Elsevier), and IEEE TRANSACTIONS ON COMMUNICATIONS. He is an Associate Editor of *International Journal of Computers and Applications* (Taylor and Francis Group).



ATHIRAH NAJIHAH ZULKIFILI received the Bachelor of Computer Science degree (Hons.) in software engineering, in 2024. She is currently pursuing the Ph.D. degree in robotics. She is skilled in programming languages, such as Python and C++; and proficient in tools, such as ROS and OpenCV. Through the Ph.D. studies, she aims to contribute to advancements in autonomous systems and AI-driven robotics, with a focus on addressing real-world challenges through automa-

tion and sustainable technology. Her research interests include artificial intelligence, computer vision, robotics applications, autonomous vehicles, path planning, and framework development.



JALAL UDDIN MD AKBAR was born in Chittagong, Bangladesh, in 1998. He received the B.S. degree (Hons.) in computer science engineering from International Islamic University Chittagong, Bangladesh, in 2021. He is currently pursuing the master's (by research) in soft computing and intelligent systems with the Universiti Malaysia Pahang Al-Sultan Abdullah (UMPSA), Pekan, Pahang, Malaysia. He has been a Research Assistant with the Soft Computing and Intelligent Systems Laboratory, UMPSA, since 2022. Based on his academic excellence, he received the prestigious Master's Research Scheme Scholarship from the Ministry of Higher Education, Malaysia. He has showcased his research work at various international conferences and forums. Through his dedication, he aims to advance innovations in artificial intelligence to create transformative solutions for challenging real-world problems. His research interests include machine learning, deep learning, and computer vision techniques for real-world applications. He is an active member of professional organizations.



LAVINDAR SINGH is a Seasoned Solution Architect and a Tech Innovator with expertise in AI, the IoT, and green energy solutions. He excels in creating impactful technologies, such as edge AI apps for biodiversity, smart agriculture systems, and sustainable energy projects. With experience in project management, software development, and smart tracking systems, he is committed to leveraging technology for environmental and societal benefits. Currently, he drives innovation in transformative digital solutions, with FPT software.



SYAFIQ FAUZI KAMARULZAMAN (Senior Member, IEEE) received the B.Eng., master's, and Ph.D. degrees from the University of Tsukuba, Japan, in 2010, 2012, and 2015, respectively. He was a fellow of the Universiti Malaysia Pahang (2012–2015). He is currently a Senior Lecturer with the Faculty of Computing, Universiti Malaysia Pahang. His research interests include control systems and artificial intelligence, which includes applications on autonomous vehicles, robotics, and the Internet of Things. He is a member of Malaysian Robotics and Automation Society. He is a certified Cisco Instructor for CCNA Program and the Internet of Things.

...